Biovision — Beaver Detection System

Product Documentation

Cover Page

Project title	Biovision — Beaver Detection System
Team members	Rebecca Huang, Xinyu Wang, Steven Liang
Instructor	Luyao Niu, Mark Licata
Industrial sponsors	WDFW, AWS
Submission date	March 13, 2026

---

1. Product Overview (Non-Technical)

What is it?
Biovision is a software tool that analyzes wildlife camera images to detect whether a beaver (or other animals) appears in each photo. It is designed for researchers, land managers, and conservation teams who need to process large numbers of trail-camera or similar images.

What problem does it solve?
Manually reviewing thousands of images for beaver presence is time-consuming and error-prone. Biovision automates this by using an AI vision model (hosted on AWS) to classify images and optionally draw boxes around beavers, producing a spreadsheet (CSV) that can be used in other workflows.

Who is it for?
- Wildlife and conservation researchers
- Land managers and environmental agencies
- Anyone with image sets (local folders or cloud storage) who needs beaver or animal detection and structured results

Primary features
- **Batch detection**: Process a single image, a folder of images, or thousands of images stored in Amazon S3.
- **Beaver classification**: Yes/no beaver presence plus a short reason and confidence score.

- **Bounding boxes**: Optional mode to draw a box around the most visible beaver in each image.
- **Animal identification**: Optional detection of animal presence and species (e.g., Beaver, Nutria, Raccoon, Deer, etc.).
- **Trail-camera overlays**: Optional reading of location/site codes from text overlays on images.
- **CSV output**: Standardized results (file path, beaver yes/no, confidence, reason, animal type, etc.) for use in spreadsheets or other tools.
- **Web and CLI**: Use from the command line, a Gradio web UI, or a Next.js chat app that answers questions about your results.

---

2. System Requirements

Supported operating systems

- **macOS** (primary development target)
- **Linux** (e.g., Ubuntu 20.04+)
- **Windows**: Supported via WSL2 or native Python; AWS CLI and credentials must be configured.

Hardware requirements

- **CPU**: Any modern x86_64 or ARM64 processor.
- **RAM**: Minimum 4 GB; 8 GB or more recommended for batch processing and the web UI.
- **Disk**: Enough space for the project, a Python environment, and output CSV files. No fixed minimum.
- **Network**: Internet connection required for AWS Bedrock (model inference) and, if used, S3 access.

Software dependencies

- **Python**: 3.11 or newer.
- **Package manager**: uv (recommended) or `pip`.
- **Node.js**: 18+ (only if using the `web/` Next.js app).
- **AWS**: An AWS account with Bedrock and (optionally) S3 access; AWS CLI or equivalent for credentials.

Internet and accounts

- **Internet**: Required for calling AWS Bedrock and for reading images from S3.
- **User accounts**:
- AWS account with permissions for Bedrock (e.g., `bedrock:InvokeModel`) and, if using S3, `s3:GetObject` (and `s3:ListBucket` for prefix listing).
- No other mandatory accounts (e.g., no separate sign-up for the tool itself).

Optional: Package contents (software-only project)

This is a software-only project. Delivered items are:

- Source code (CLI, Gradio UI, and optional Next.js web app).
- Configuration examples (e.g., `.env.example`).
- This documentation.

There are no physical components or packages to list.

---

3. Installation and Setup

Follow these steps exactly for a first-time setup.

Step 1 — Install Python and uv

1. Install **Python 3.11 or newer** from python.org or your system package manager.
2. Install **uv** (recommended):
   `bash curl -LsSf https://astral.sh/uv/install.sh | sh`
   Or follow uv's documentation.

Step 2 — Clone or download the project

1. Open a terminal.
2. Go to the folder where you want the project (e.g., `cd ~/Projects`).
3. If you have git:
   `git clone https://github.com/GIX-Luyao/final-codebase-biovision.git`
   then:
   `cd final-codebase-biovision`
4. If you received a ZIP, unzip it and `cd` into the project folder.

Step 3 — Create a virtual environment and install the CLI

Run:

```
 uv venv
source .venv/bin/activate   # On Windows: .venv\Scripts\activate
uv pip install -e .
```

You should see the environment activate and packages install without errors.

Step 4 — Configure AWS credentials

1. Install and configure the AWS CLI (see AWS CLI configuration).

2. Run `aws configure` and enter your Access Key ID, Secret Access Key, and default region (e.g., `us-east-2`).
3. Ensure your IAM user/role has permission to call Bedrock (e.g., `bedrock:InvokeModel`) and, if you use S3, to read from the chosen bucket/prefix.

Step 5 — Configure the application (environment file)

1. In the project root, copy the example env file:
   `cp .env.example .env`
2. Open `.env` in a text editor.
3. Set at least:
4. `AWS_REGION` (e.g., `us-east-2`)
5. `AWS_PROFILE` (e.g., `default`)
6. `BEAVER_BEDROCK_MODEL_ID` — your Bedrock model or inference profile ARN (see README or your AWS setup).
7. Save the file. The CLI and Gradio UI load `.env` automatically.

Step 6 — Verify the CLI

Run:

```
beaver-id --help
```

You should see the help text. To test on a single image (replace with your image path):

```
beaver-id /path/to/one/image.jpg --output test_results.csv
```

Optional — Gradio UI

From the same activated environment:

```
beaver-id-ui
```

Then open the URL shown in the terminal (e.g., `http://127.0.0.1:7860`) in a browser. No extra installation steps beyond the main project.

Optional — Next.js web app (chat over CSV)

1. `cd web`
2. `npm install`
3. Create `web/.env.local` with at least:
   `BEAVER_CHAT_CSV=/absolute/path/to/your/beaver_results.csv`
   and the same `AWS_REGION`, `AWS_PROFILE`, and Bedrock model ID as above.

4. `npm run dev`
5. Open the URL shown (e.g., `http://localhost:3000`) and use the chat to ask questions about your CSV (e.g., "How many beavers?" or "List animals in this CSV").

---

4. Operating Instructions

Starting the system

- **CLI**: Open a terminal, activate the environment (`source .venv/bin/activate`), then run `beaver-id` with your input path and options.
- **Gradio UI**: Run `beaver-id-ui` , then open the printed URL in a browser.
- **Web chat**: From `web/` , run `npm run dev` and open the printed URL.

Performing primary tasks

Run batch beaver detection (CLI)

- Single image:

```
beaver-id /path/to/image.jpg --output results.csv
```

- Directory:

```
beaver-id /path/to/images/folder --output results.csv
```

- S3 prefix:

```
beaver-id s3://bucket-name/prefix/ --output results.csv --max-workers 6
```

Bounding box mode (CLI)

```
beaver-id /path/to/images --task bbox --output beaver_bbox.csv
```

Using the Gradio UI

- **Upload Image**: Upload a file or enter an S3 path, choose task (classify/bbox), then click Run. View the JSON result and, for bbox, the preview image.
- **S3 Batch / Batch Upload**: Enter an S3 prefix or upload multiple files, set workers and batch size, then Run Batch. Download the generated CSV when done.
- **Chat**: Point to a CSV path and ask questions such as "How many deer?" or "List animals."

Shutting down

- **CLI**: It exits when the run finishes.
- **Gradio**: Close the browser tab; in the terminal press `Ctrl+C` to stop the server.
- **Next.js**: In the terminal press `Ctrl+C` to stop the dev server.

---

5. Technical Specifications

Architecture

- **CLI** (`src/beaver_id/cli.py`): Entrypoint that discovers images (local or S3), resizes/compresses them for the API, calls AWS Bedrock with vision prompts, and writes CSV. Concurrency via `ThreadPoolExecutor` .
- **Gradio app** (`src/beaver_id/gradio_app.py`): Web UI that reuses the same detection logic for single-image, S3 single, S3 batch, and local batch, plus a simple CSV-based Q&A.
- **Web app** (`web/`): Next.js app with a chat interface that reads a single CSV and answers natural-language questions about animal counts and types using the Vercel AI SDK and Bedrock (or configured backend).

Technologies

- **Language**: Python 3.11+.
- **Key libraries**: `boto3` (AWS), `python-dotenv` , `Pillow` (image handling), `gradio` .
- **Frontend (web)**: Next.js 16, React 19, Tailwind CSS, TypeScript.
- **Cloud**: AWS Bedrock (vision model inference), optional AWS S3 for image input/output.

Model and inference

- **Default model**: Anthropic Claude (Opus) via AWS Bedrock inference profile (e.g., `us.anthropic.claude-opus-4-5-20251101-v1:0`).
- **Tasks**: (1) Beavers: classify (yes/no + reason + confidence) or bbox (one box per image). (2) Optional animal detection (species from a fixed list). (3) Optional overlay text extraction (location/site codes).
- **Image handling**: Images are resized (max dimension configurable, default 2048), then JPEG-compressed to stay under the Bedrock payload limit (e.g., 5 MB).

Database and integrations

- **No database required** for the core CLI or Gradio UI; results are written to CSV files.
- **Optional**: The `.env.example` references a Postgres URL and API base URL for possible future job or DB integration; these are not required for basic operation.
- **Integrations**: AWS Bedrock (required), AWS S3 (optional, for S3 image inputs).

Output schema (CSV)

Columns include: `image_path` , `has_beaver` , `confidence` , `reason` , `bbox` (JSON if bbox task), `has_animal` , `animal_type` , `animal_group` , `animal_confidence` , `animal_reason` , `overlay_location` , `overlay_confidence` , `overlay_reason` , `model_id` , `exif_timestamp` , `exif_location` , `error` .

---

6. Data Privacy, Security, and Limitations

Data privacy

- **Image data**: Images are sent to AWS Bedrock for inference. Only the image bytes and the prompts are transmitted; no built-in logging of image content beyond what AWS may record per your account's policies.
- **CSV results**: Stored only where you choose (local disk or S3). The application does not upload result CSVs to third parties.
- **Recommendation**: Do not process images containing personal data or sensitive locations unless your use is compliant with your organization's and AWS's data policies.

Security

- **Credentials**: Use IAM roles or AWS credentials (e.g., `aws configure`) with minimal required permissions (Bedrock invoke, S3 read if needed). Do not commit `.env` or secrets to version control.
- **Network**: All Bedrock and S3 traffic uses HTTPS.
- **Dependencies**: Keep Python and Node dependencies up to date and review for known vulnerabilities.

Known operational limitations

- **Model accuracy**: Detection depends on the underlying vision model; false positives and false negatives are possible, especially for partial, distant, or low-quality images.
- **Rate limits**: Bedrock has account/region rate limits; large batches may need tuning of `--max-workers` and batch size to avoid throttling.
- **Image size**: Large images are downscaled and compressed; very small or heavily compressed images may reduce accuracy.
- **Single bbox**: In bbox mode, only one bounding box per image is produced (the "most visible" beaver).
- **Chat**: The Gradio and Next.js chat features are simple rule-based or small-scope Q&A over the CSV (e.g., counts, animal list); they are not full general-purpose chatbots.

---

7. Troubleshooting

Symptom	What to check
`Permission denied` or 403 when pushing/pulling	GitHub or AWS permissions; ensure AWS credentials and Bedrock/S3 permissions are correct.
`No images found for input`	Verify path: local path must exist; S3 path must use `s3://bucket/prefix/` and the bucket/prefix must contain supported image files (e.g., .jpg, .png).

Symptom	What to check
`ModuleNotFoundError` or `beaver-id: command not found`	Activate the virtual environment (`source .venv/bin/activate`) and ensure you ran `uv pip install -e .` from the project root.
Bedrock throttling or timeout	Reduce `--max-workers` or batch size; add retries (see README: `BEAVER_BEDROCK_MAX_RETRIES` , backoff).
Gradio/Next.js won't start	Check that the port is free; try another port if needed. For Next.js, ensure `npm install` was run in `web/` .
Empty or wrong CSV	Confirm the model ID and region in `.env` ; check that images are valid and readable; look at the `error` column in the CSV for per-image failures.
Chat says "No CSV rows" or "No animals"	Ensure `BEAVER_CHAT_CSV` (or the CSV path in the UI) points to an existing CSV with the expected columns (e.g., `animal_type`).

---

8. Limitations

- **Known constraints**:
- Depends on AWS Bedrock availability and your account's quotas.
- Animal species are limited to a fixed list in the prompt (e.g., Beaver, Nutria, Raccoon, Deer, etc.); "other mammal" / "other bird" used for unknowns.
- Overlay reading is optional and tuned for specific overlay formats; accuracy may vary.
- **Performance**:
- Throughput is limited by Bedrock API latency and rate limits; batch run time grows roughly linearly with image count and worker count.
- No GPU required; all inference is remote.
- **Unfinished / future work**:
- Retries and backoff for Bedrock are present but further metrics/logging for batch runs are noted as next steps in the README.
- Optional DB and job-queue integration (e.g., Postgres, API base URL) are referenced in env examples but not required or fully documented for this release.

End of document. Fill in the cover-page placeholders (team members, instructor, sponsor, submission date, and logo if desired) before generating the final PDF.